esprit Se former autrement HONORIS UNITED UNIVERSITIES	Année Universitaire : 2025-2026 **Applications Web Distribuées**

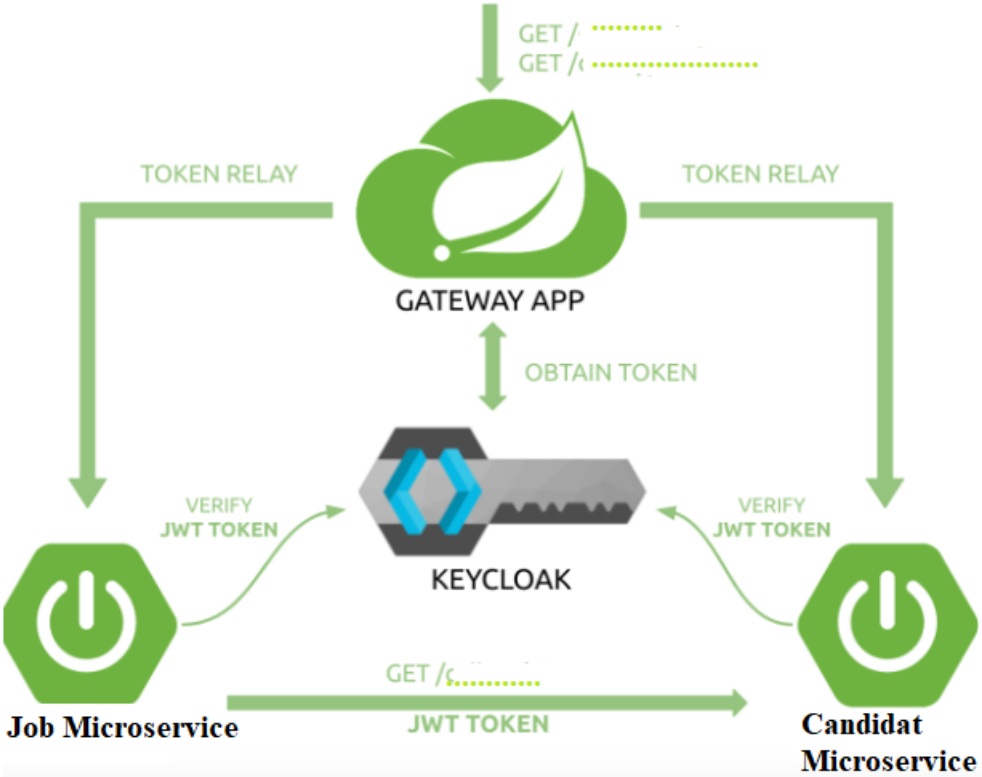
Sécurisation d'une API Gateway avec Keycloak

Objectif

- Assurer la sécurité d'une API Gatewaye en utilisant Auth2.0.
- Assurer la sécurité de l'accès vers l'ensemble des microservices.

Principe de fonctionnement

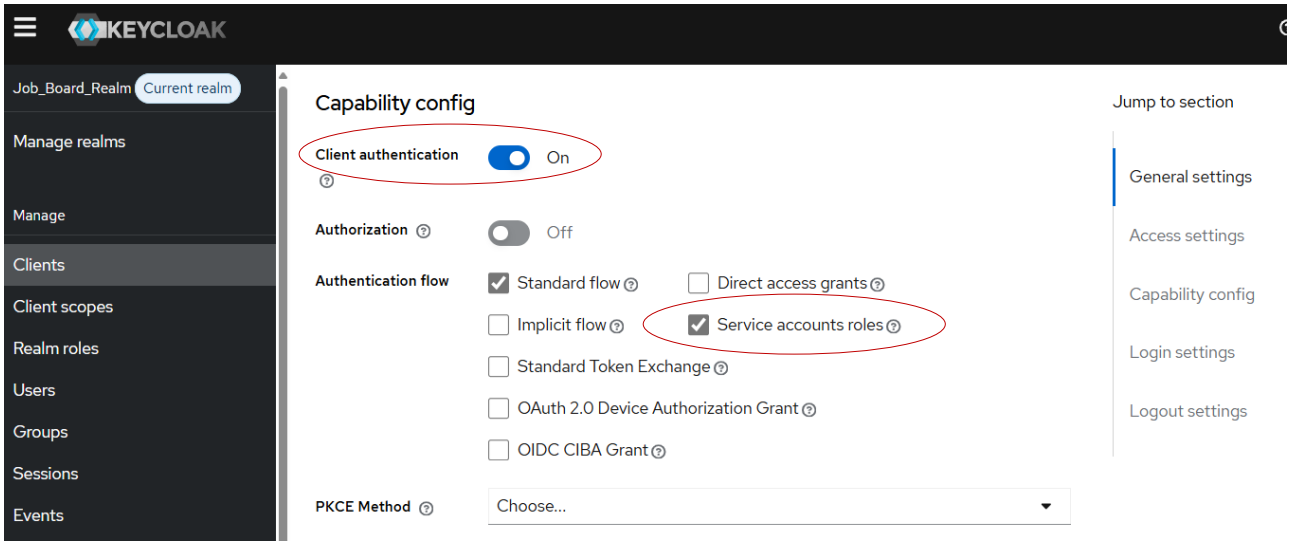
Lors de l'accès aux Microservices à travers le Gateway, celle-ci va renvoyer automatiquement l'utilisateur vers Keycloak pour récupérer un Token. Keycloak authentifiera l'utilisateur si besoin, puis renverra des informations sur l'utilisateur et le fameux Token à notre Viewer. Ce Token sera ensuite utilisé pour l'accès vers le microservice en question.



Les étapes à suivre

1. Créer un realm « **JobBoardKeycloack** »
2. Créer un client **gateway**
   - clientID : gateway (nom de l'application Microservice Gateway)
   - RootURL, HomeURL et AdminURL : http://localhost:8093 (Port du Gateway)

➢ Vérifier la configuration ajoutée selon la figure ci-dessous :



3. Dans le fichier pom.xml de votre projet Gateway, ajouter le contenu suivant :

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

NB :

- L'API Gateway doit **uniquement valider les tokens** fournis par Keycloak, alors **Spring Boot Starter OAuth2 Resource Server** suffit. Cette bibliothèque Spring Boot supporte **OpenID Connect** et **OAuth2**, qui sont les protocoles utilisés par Keycloak.

4. Sous le projet Gateway, ajouter la classe **SecurityConfig**

```
import org.springframework.context.annotation.*;
import org.springframework.security.config.Customizer;
import
org.springframework.security.config.annotation.web.reactive.EnableWebFluxSecurity;
import org.springframework.security.config.web.server.ServerHttpSecurity;
import org.springframework.security.web.server.SecurityWebFilterChain;


@Configuration
```

```
@EnableWebFluxSecurity
 public class SecurityConfig {

  @Bean
  public SecurityWebFilterChain securityWebFilterChain(ServerHttpSecurity HttpSecurity)
{
        return serverHttpSecurity.csrf(ServerHttpSecurity.CsrfSpec::disable)
                .authorizeExchange(exchange -> exchange.pathMatchers("/eureka/**")
                        .permitAll()
                        .anyExchange().authenticated()
                ).oauth2ResourceServer((oauth) -> oauth
                        .jwt(Customizer.withDefaults()))

                .build();
    }
}
```

5. Dans le fichier **application.properties**, ajouter la propriété suivante :

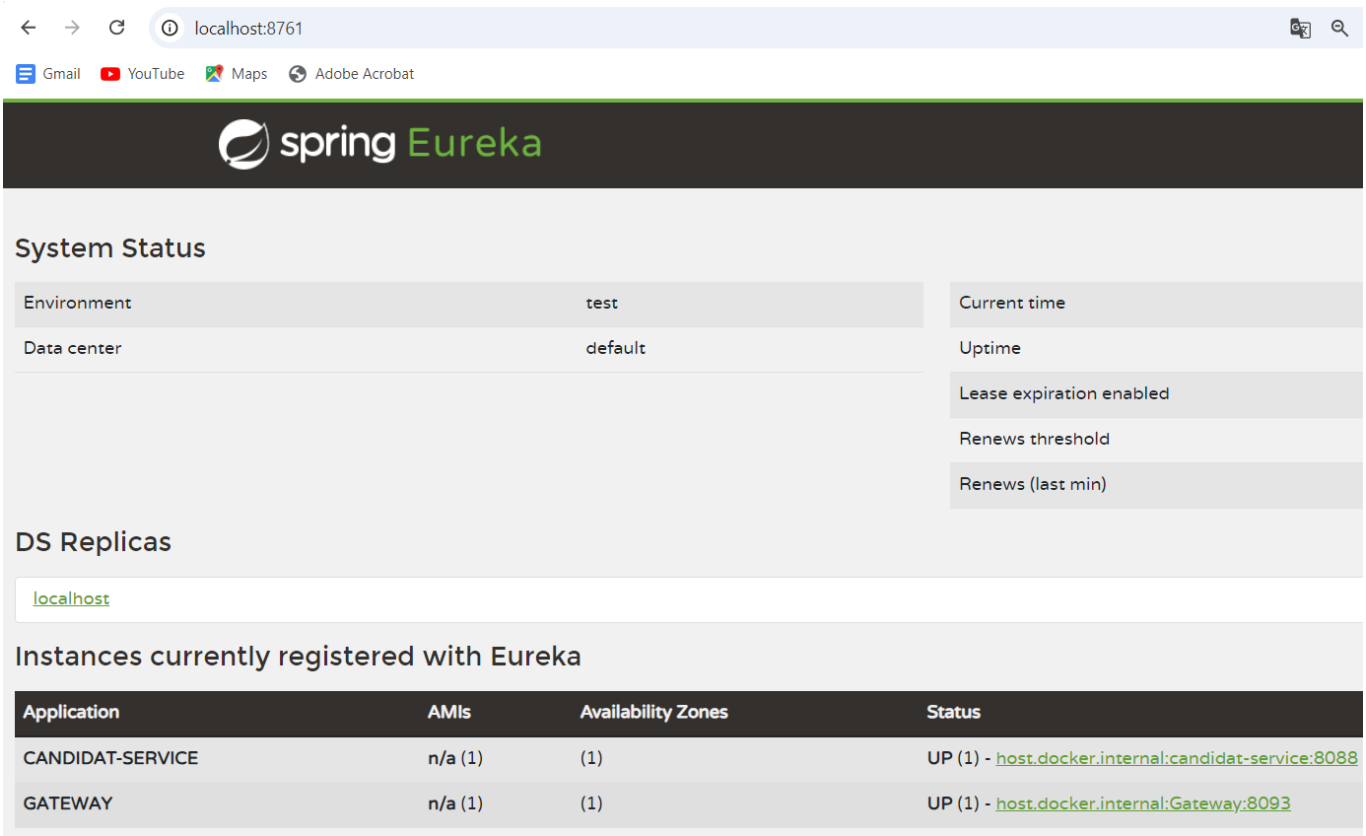
```
spring.security.oauth2.resourceserver.jwt.issuer-uri=
http://localhost:8080/realms/JobBoardKeycloack
```

```
spring.application.name=gateway
server.port=8093

# eureka registration
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
eureka.client.register-with-eureka=true

spring.security.oauth2.resourceserver.jwt.issuer-uri=http://localhost:8080/realms/JobBoardKeycloack
```

6. Maintenant, nous allons tester l'accès sécurisé vers le MS de G. Candidats.
Exécuter les projets :

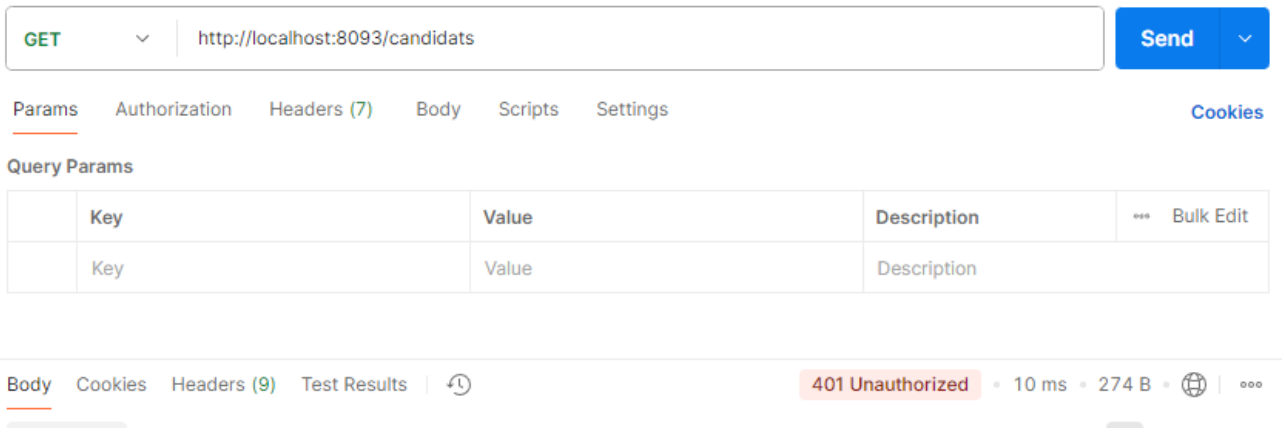
1. Eureka
2. Candidat
3. Gateway

7. Accéder au serveur eureka pour vérifier l'enregistrement du Gateway et Candidat:



8. Accès au micro-service candidat via Gateway

Accès <u>Sans la clé de sécurité</u>



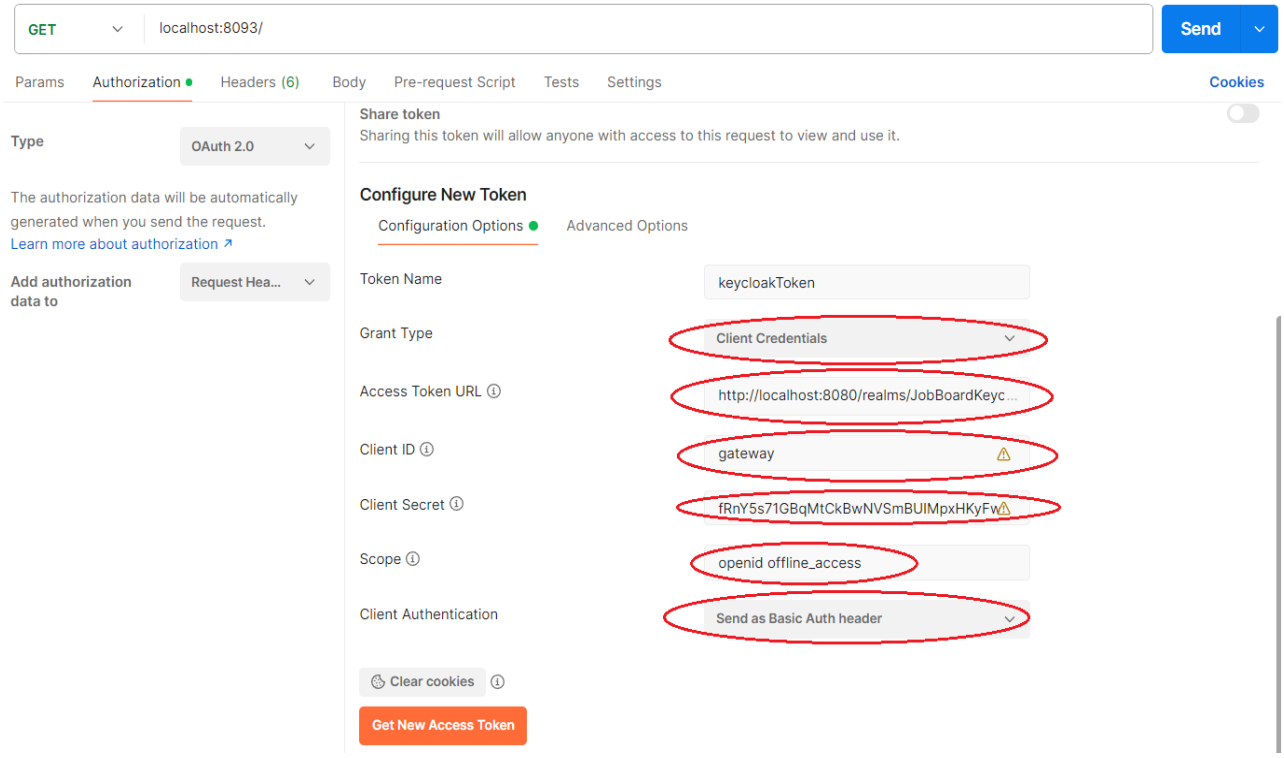
NB :

- **8093** : est le port de l'API Gateway
- **/candidats** : Le path d'accès au MS de Candidat

Génération de la clé de sécurité

Sur postman, nous allons générer un token à travers Keycloak

- Veuillez choisir Authorization-> OAuth 2.0
- Veuillez configurer un nouveau token

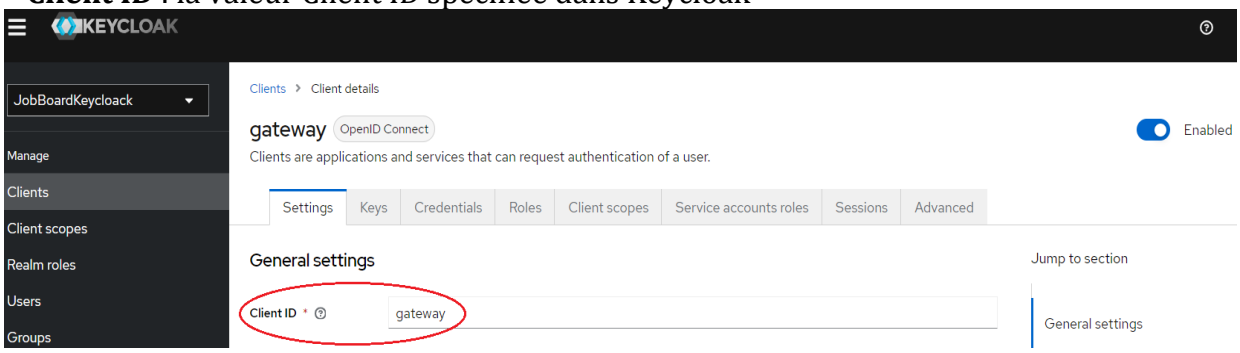


Comme le montre la figure ci-dessus :

- **Token Name** : veuillez choisir KeycloakToken
- **Grant type** : Client Credentials
- **Access Token URL** : Pour récupérer cette valeur à partir de l'interface de Keycloak : **Realm Settings → General → Endpoints**, puis cliquer sur le lien « OpenID EndPoint Configuration ». Récupérer ensuite la valeur de "**token_endpoint**" :

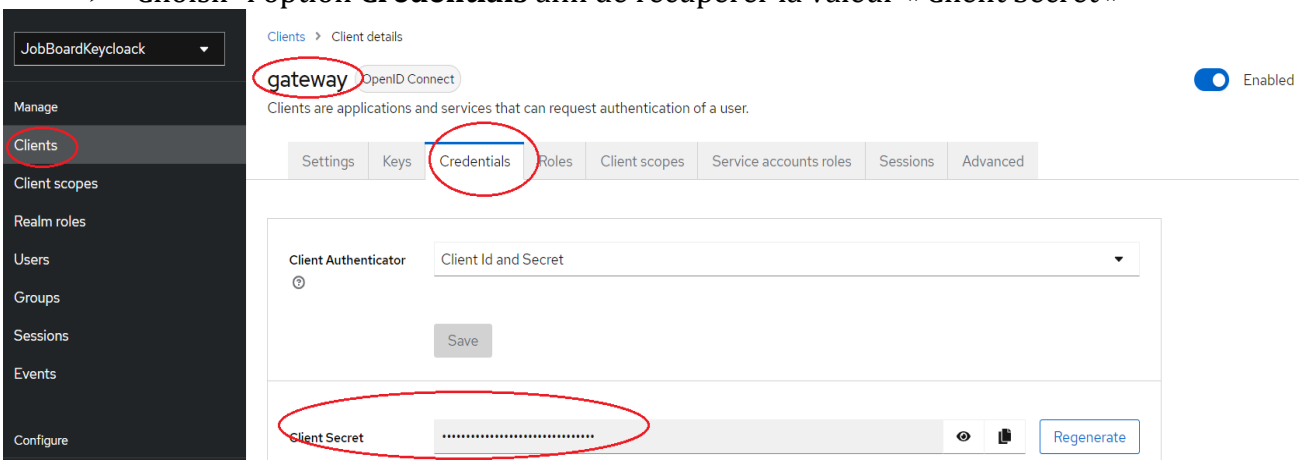
```
{"issuer":"http://localhost:8080/realms/JobBoardKeycloack","authorization_endpoint":"http://localhost:8080/realms/JobBoardKeycloack/protocol/openid-connect/auth","token_endpoint":"http://localhost:8080/realms/JobBoardKeycloack/protocol/openid-connect/token","introspection_endpoint":"http://localhost:8080/realms/JobBoardKeycloack/protocol/openid-
```


- **Client ID** : la valeur Client ID spécifiée dans Keycloak



- **Client secret** :
  Au niveau du Client « gateway » :

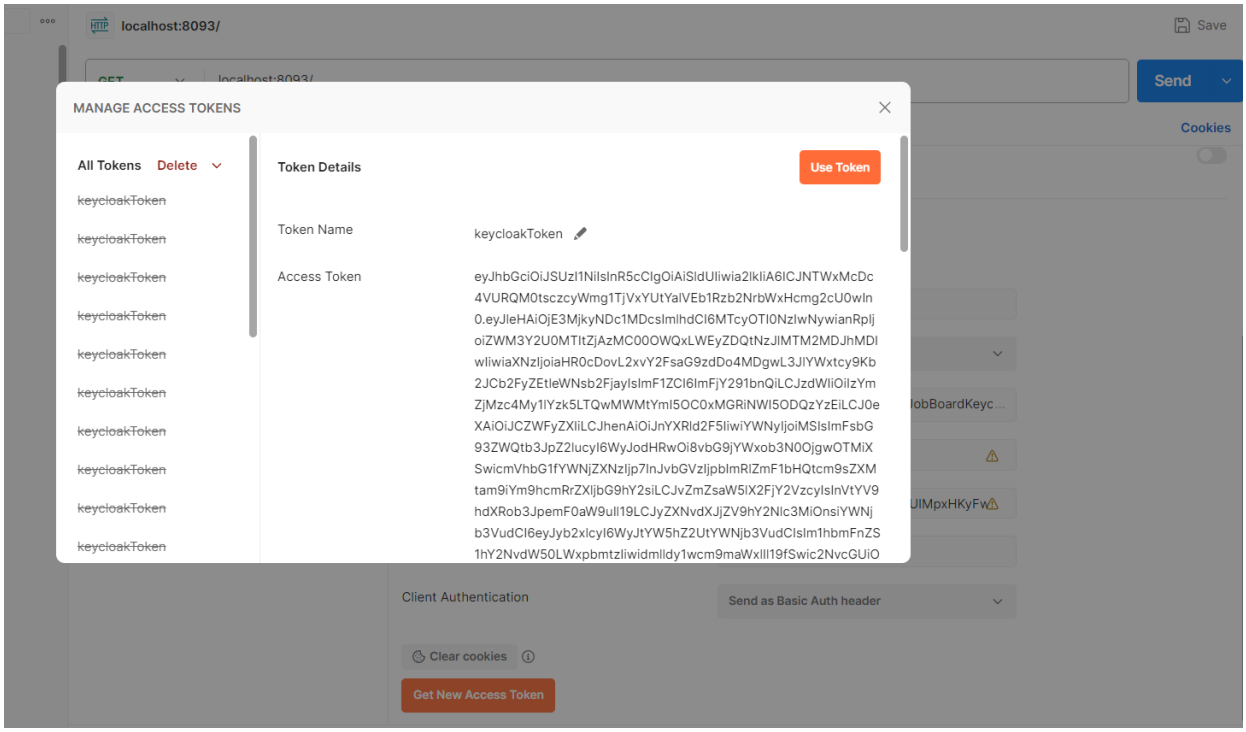
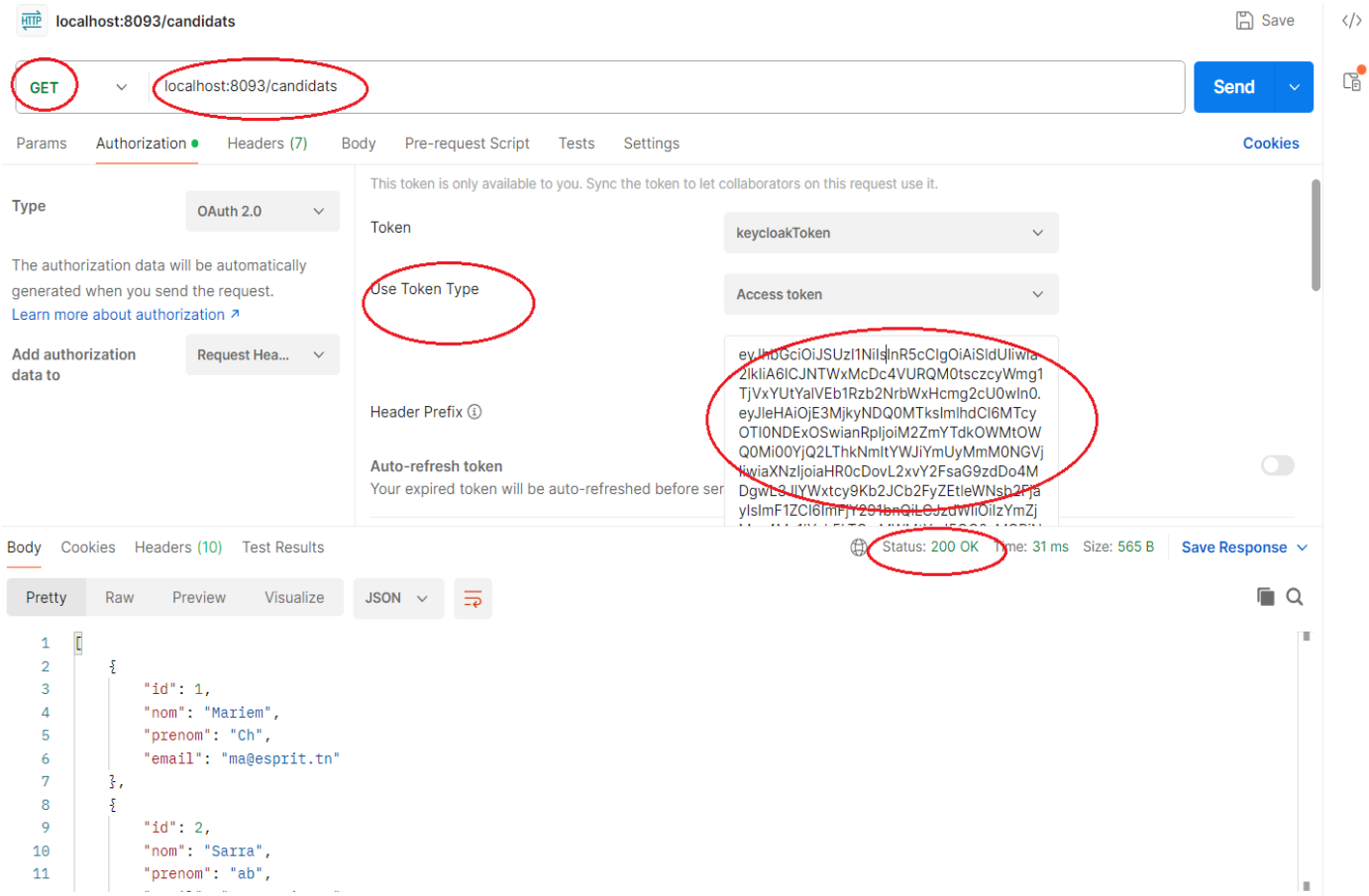
  - Choisir l'option **Credentials** afin de récupérer la valeur « Client Secret »



- **Scope** : openid offline_access (pour obtenir à la fois : un **ID Token** pour prouver l'identité de l'utilisateur et un **Refresh Token** pour un accès continu même après la fin de la session utilisateur)

- **Client Authentication** : Send a Basic Auth header

- Cliquer sur Get New Access Token. Vous pouvez voir la Valeur de token générée.



- Tester l'accès vers le Microservice Candidat comme suit :